

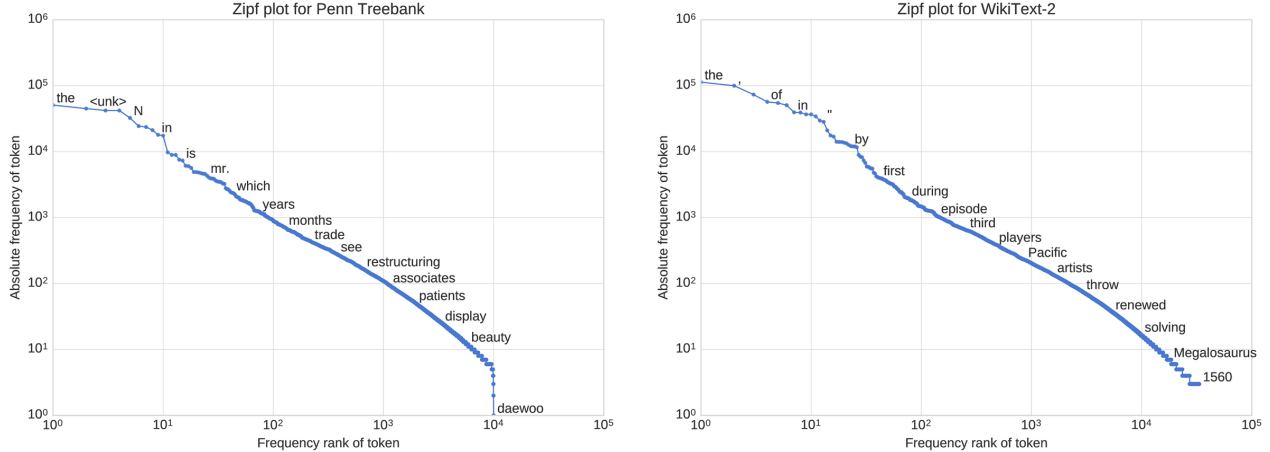


Figure 3. Zipfian plot over the training partition in Penn Treebank and WikiText-2 datasets. Notice the severe drop on the Penn Treebank when the vocabulary hits 10^4 . Two thirds of the vocabulary in WikiText-2 are past the vocabulary cut-off of the Penn Treebank.

language modeling training schemes, $k_1 = k_2$, meaning that every k timesteps truncated BPTT is performed for the k previous timesteps. This results in only a single RNN output receiving backpropagation for k timesteps, with the other extreme being that the first token receives backpropagation for 0 timesteps. This issue is compounded by the fact that most language modeling code split the data temporally such that the boundaries are always the same. As such, most words in the training data will never experience a full backpropagation for k timesteps.

In our task, the pointer component always looks L timesteps into the past if L past timesteps are available. We select $k_1 = 1$ and $k_2 = L$ such that for each timestep we perform backpropagation for L timesteps and advance one timestep at a time. Only the loss for the final predicted word is used for backpropagation through the window.

5.2. Model Details

Our experimental setup reflects that of Zaremba et al. (2014) and Gal (2015). We increased the number of timesteps used during training from 35 to 100, matching the length of the window L . Batch size was increased to 32 from 20. We also halve the learning rate when validation perplexity is worse than the previous iteration, stopping training when validation perplexity fails to improve for three epochs or when 64 epochs are reached. The gradients are rescaled if their global norm exceeds 1 (Pascanu et al., 2013b).³ We evaluate the medium model configuration which features a hidden size of $H = 650$ and a two layer LSTM. We compare against the large model configu-

³The highly aggressive clipping is likely due to the increased BPTT length. Even with such clipping early batches may experience excessively high perplexity, though this settles rapidly.

ration which features a hidden size of 1500 and a two layer LSTM.

We produce results for two model types, an LSTM model that uses dropout regularization and the pointer sentinel-LSTM model. The variants of dropout used were zoneout (Krueger et al., 2016) and variational inference based dropout (Gal, 2015). Zoneout, which stochastically forces some recurrent units to maintain their previous values, was used for the recurrent connections within the LSTM. Variational inference based dropout, where the dropout mask for a layer is locked across timesteps, was used on the input to each RNN layer and also on the output of the final RNN layer. We used a value of 0.5 for both dropout connections.

5.3. Comparison over Penn Treebank

Table 2 compares the pointer sentinel-LSTM to a variety of other models on the Penn Treebank dataset. The pointer sentinel-LSTM achieves the lowest perplexity, followed by the recent Recurrent Highway Networks (Zilly et al., 2016). The medium pointer sentinel-LSTM model also achieves lower perplexity than the large LSTM models. Note that the best performing large variational LSTM model uses computationally intensive Monte Carlo (MC) dropout averaging. Monte Carlo dropout averaging is a general improvement for any sequence model that uses dropout but comes at a greatly increased test time cost. In Gal (2015) it requires rerunning the test model with 1000 different dropout masks. The pointer sentinel-LSTM is able to achieve these results with far fewer parameters than other models with comparable performance, specifically with less than a third the parameters used in the large variational LSTM models.

We also test a variational LSTM that uses zoneout, which

serves as the RNN component of our pointer sentinel-LSTM mixture. This variational LSTM model performs BPTT for the same length L as the pointer sentinel-LSTM, where $L = 100$ timesteps. The results for this model ablation are worse than that of Gal (2015)’s variational LSTM without Monte Carlo dropout averaging.

5.4. Comparison over WikiText-2

As WikiText-2 is being introduced in this dataset, there are no existing baselines. We provide two baselines to compare the pointer sentinel-LSTM against: our variational LSTM using zoneout and the medium variational LSTM used in Gal (2015).⁴ Attempts to run the Gal (2015) large model variant, a two layer LSTM with hidden size 1500, resulted in out of memory errors on a 12GB K80 GPU, likely due to the increased vocabulary size. We chose the best hyperparameters from PTB experiments for all models.

Table 3 shows a similar gain made by the pointer sentinel-LSTM over the variational LSTM models. The variational LSTM from Gal (2015) again beats out the variational LSTM used as a base for our experiments.

6. Analysis

6.1. Impact on Rare Words

A hypothesis as to why the pointer sentinel-LSTM can outperform an LSTM is that the pointer component allows the model to effectively reproduce rare words. An RNN may be able to better use the hidden state capacity by deferring to the pointer component. The pointer component may also allow for a sharper selection of a single word than may be possible using only the softmax.

Figure 4 shows the improvement of perplexity when comparing the LSTM to the pointer sentinel-LSTM with words split across buckets according to frequency. It shows that the pointer sentinel-LSTM has stronger improvements as words become rarer. Even on the Penn Treebank, where there is a relative absence of rare words due to only selecting the most frequent 10k words, we can see the pointer sentinel-LSTM mixture model provides a direct benefit.

While the improvements are largest on rare words, we can see that the pointer sentinel-LSTM is still helpful on relatively frequent words. This may be the pointer component directly selecting the word or through the pointer supervision signal improving the RNN by allowing gradients to flow directly to other occurrences of the word in that window.

⁴<https://github.com/yaringal/BayesianRNN>

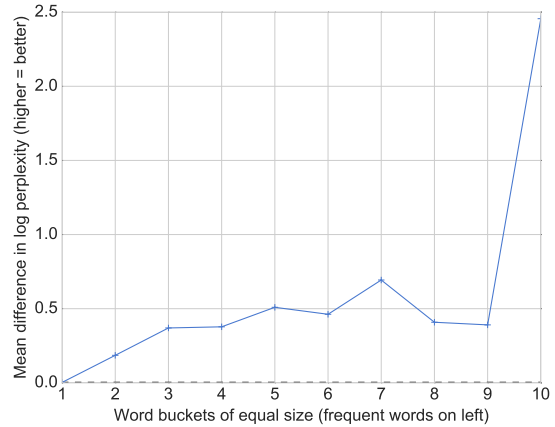


Figure 4. Mean difference in log perplexity on PTB when using the pointer sentinel-LSTM compared to the LSTM model. Words were sorted by frequency and split into equal sized buckets.

6.2. Qualitative Analysis of Pointer Usage

In a qualitative analysis, we visualized the gate use and pointer attention for a variety of examples in the validation set, focusing on predictions where the gate primarily used the pointer component. These visualizations are available in the supplementary material.

As expected, the pointer component is heavily used for rare names such as *Seidman* (23 times in training), *Iverson* (7 times in training), and *Rosenthal* (3 times in training).

The pointer component was also heavily used when it came to other named entity names such as companies like *Honeywell* (8 times in training) and *Integrated* (41 times in training, though due to lowercasing of words this includes *integrated circuits*, *fully integrated*, and other generic usage).

Surprisingly, the pointer component was also used for many frequent tokens. For selecting the unit of measurement (tons, kilograms, ...) or the short scale of numbers (thousands, millions, billions, ...), the pointer would refer to previous recent usage. This is to be expected, especially when phrases are of the form *increased from N tons to N tons*. The model can even be found relying on a mixture of the softmax and the pointer for predicting certain frequent verbs such as *said*.

Finally, the pointer component can be seen pointing to words at the very end of the 100 word window (position 97), a far longer horizon than the 35 steps that most language models truncate their backpropagation training to. This illustrates why the gating function must be integrated into the pointer component. If the gating function could only use the RNN hidden state, it would need to be wary of words that were near the tail of the pointer, especially if it was not able to accurately track exactly how long it

Model	Parameters	Validation	Test
Mikolov & Zweig (2012) - KN-5	2M [‡]	—	141.2
Mikolov & Zweig (2012) - KN5 + cache	2M [‡]	—	125.7
Mikolov & Zweig (2012) - RNN	6M [‡]	—	124.7
Mikolov & Zweig (2012) - RNN-LDA	7M [‡]	—	113.7
Mikolov & Zweig (2012) - RNN-LDA + KN-5 + cache	9M [‡]	—	92.0
Pascanu et al. (2013a) - Deep RNN	6M	—	107.5
Cheng et al. (2014) - Sum-Prod Net	5M [‡]	—	100.0
Zaremba et al. (2014) - LSTM (medium)	20M	86.2	82.7
Zaremba et al. (2014) - LSTM (large)	66M	82.2	78.4
Gal (2015) - Variational LSTM (medium, untied)	20M	81.9 ± 0.2	79.7 ± 0.1
Gal (2015) - Variational LSTM (medium, untied, MC)	20M	—	78.6 ± 0.1
Gal (2015) - Variational LSTM (large, untied)	66M	77.9 ± 0.3	75.2 ± 0.2
Gal (2015) - Variational LSTM (large, untied, MC)	66M	—	73.4 ± 0.0
Kim et al. (2016) - CharCNN	19M	—	78.9
Zilly et al. (2016) - Variational RHN	32M	72.8	71.3
Zoneout + Variational LSTM (medium)	20M	84.4	80.6
Pointer Sentinel-LSTM (medium)	21M	72.4	70.9

Table 2. Single model perplexity on validation and test sets for the Penn Treebank language modeling task. For our models and the models of Zaremba et al. (2014) and Gal (2015), medium and large refer to a 650 and 1500 units two layer LSTM respectively. The medium pointer sentinel-LSTM model achieves lower perplexity than the large LSTM model of Gal (2015) while using a third of the parameters and without using the computationally expensive Monte Carlo (MC) dropout averaging at test time. Parameter numbers with [‡] are estimates based upon our understanding of the model and with reference to Kim et al. (2016).

Model	Parameters	Validation	Test
Variational LSTM implementation from Gal (2015)	20M	101.7	96.3
Zoneout + Variational LSTM	20M	108.7	100.9
Pointer Sentinel-LSTM	21M	84.8	80.8

Table 3. Single model perplexity on validation and test sets for the WikiText-2 language modeling task. All compared models use a two layer LSTM with a hidden size of 650 and the same hyperparameters as the best performing Penn Treebank model.

was since seeing a word. By integrating the gating function into the pointer component, we avoid the RNN hidden state having to maintain this intensive bookkeeping.

7. Conclusion

We introduced the pointer sentinel mixture model and the WikiText language modeling dataset. This model achieves state of the art results in language modeling over the Penn Treebank while using few additional parameters and little additional computational complexity at prediction time.

We have also motivated the need to move from Penn Treebank to a new language modeling dataset for long range dependencies, providing WikiText-2 and WikiText-103 as potential options. We hope this new dataset can serve as a platform to improve handling of rare words and the usage of long term dependencies in language modeling.

References

- Adi, Yossi, Kermany, Einat, Belinkov, Yonatan, Lavi, Ofer, and Goldberg, Yoav. Fine-grained Analysis of Sentence Embeddings Using Auxiliary Prediction Tasks. *arXiv preprint arXiv:1608.04207*, 2016.
- Ahn, Sungjin, Choi, Heeyoul, Pärnamaa, Tanel, and Bengio, Yoshua. A Neural Knowledge Language Model. *CoRR*, abs/1608.00318, 2016.
- Bahdanau, Dzmitry, Cho, Kyunghyun, and Bengio, Yoshua. Neural Machine Translation by Jointly Learning to Align and Translate. In *ICLR*, 2015.
- Chelba, Ciprian, Mikolov, Tomas, Schuster, Mike, Ge, Qi, Brants, Thorsten, Koehn, Phillipp, and Robinson, Tony. One Billion Word Benchmark for Measuring Progress in Statistical Language Modeling. *arXiv preprint arXiv:1312.3005*, 2013.
- Cheng, Jianpeng, Dong, Li, and Lapata, Mirella. Long